

ARQUITECTURA DE SOFTWARE

Universidad Peruana Los Andes • Facultad de Ingeniería • Mg. Ing. Raúl Fernández Bejarano • 2025

1.1 DEFINICIÓN

La **arquitectura de software** es la estructura fundamental de un sistema informático, conformada por sus componentes, las relaciones entre ellos y los principios que guían su diseño y evolución.

Norma **ISO/IEC/IEEE 42010:2011**: integra comunicación entre componentes, elección de tecnologías y mecanismos para requisitos funcionales y no funcionales.

Sirve como **modelo de alto nivel** que orienta la toma de decisiones estratégicas: escalabilidad, seguridad, desempeño, interoperabilidad y mantenibilidad.

1.2 DISEÑO vs. ARQUITECTURA

Arquitectura	Diseño
Visión global del sistema	Implementación detallada de módulos
Nivel abstracto y estratégico (qué y cómo)	Nivel táctico (cómo exactamente)
Alto costo de cambio si hay errores	Más fácil de modificar en codificación

1.3 ROL DEL ARQUITECTO

- Definición inicial:** Identificar componentes, estilos y patrones.
- Alineación con el negocio:** Arquitectura soporte los objetivos organizacionales.
- Gestión de calidad:** Seguridad, rendimiento, usabilidad, escalabilidad.
- Comunicación:** Explicar la arquitectura a equipos y stakeholders.
- Evolución:** Adaptar ante cambios y nuevas tecnologías.

1.4 IMPACTO EN CALIDAD Y EFICIENCIA

- Eficiencia:** Reduce complejidad, facilita división del trabajo y reutilización.
- Mantenibilidad:** La modularidad facilita corrección de errores e incorporación de funcionalidades.
- Escalabilidad:** El sistema puede crecer sin pérdida de desempeño.
- Seguridad:** Incorporación temprana de mecanismos previene vulnerabilidades críticas.
- Sostenibilidad:** Estructura flexible y adaptable prolonga la vida útil del software.

2.1 SEPARACIÓN DE RESPONSABILIDADES

Cada componente, módulo o capa debe tener una **función claramente delimitada**. Facilita claridad, mantenimiento y escalabilidad.

Beneficio: los equipos trabajan en paralelo en diferentes partes sin afectar otras capas.

2.2 COHESIÓN Y ACOPLAMIENTO

- Cohesión:** grado en que las funciones de un módulo están relacionadas. **Alta cohesión** = módulo con objetivo específico.
- Acoplamiento:** grado de dependencia entre módulos. **Bajo acoplamiento** = módulos independientes, fáciles de reutilizar.
- Arquitectura ideal: **Alta cohesión + Bajo acoplamiento**.

EJEMPLOS (ABP)

Ejemplo 1 — Sistema de historias clínicas electrónicas

Situación: Usan archivos en papel → pérdida de información.

ABP: Definir arquitectura (componentes, relaciones, principios) para garantizar seguridad y disponibilidad.

Resultado: Documento arquitectónico con estructura en 3 capas (presentación, negocio, datos).

Ejemplo 2 — App móvil de pedidos de comida

Situación: El equipo confunde arquitectura con lógica de programación.

ABP: Arquitectura general (microservicios: pedidos, pagos, usuarios) + diseño detallado del módulo de pagos.

Resultado: Comparación práctica arquitectura vs. diseño.

Ejemplo 3 — Sistema de gestión académica universitaria

Situación: Cambios constantes en requisitos de matrícula.

ABP: Rol de arquitecto: modelo escalable + reuniones con stakeholders.

Resultado: Plan de evolución arquitectónica documentado.

CAPACIDAD Y DESEMPEÑO • Semana 01

CAPACIDAD: Explica los Fundamentos de la Arquitectura de Software, utilizando los estándares internacionales, para la producción del software.

DESEMPEÑO: Analiza los conceptos fundamentales de la arquitectura de software, reconociendo su importancia en el desarrollo de sistemas eficientes.

ARQUITECTURA EN CAPAS (Ejemplo)

CAPA DE PRESENTACIÓN

CAPA DE NEGOCIO / LÓGICA

CAPA DE DATOS

GLOSARIO DE CONCEPTOS CLAVE

Concepto	Descripción
Componente	Unidad modular con función específica
Módulo	Agrupación lógica de funcionalidades relacionadas
Patrón	Solución reutilizable a problemas frecuentes
Atributo de calidad	Seguridad, escalabilidad, rendimiento, usabilidad
Stakeholder	Actor con interés en el sistema (cliente, dev, usuario)
Microservicio	Servicio pequeño e independiente dentro del sistema

EJERCICIOS PROPUESTOS

Ejercicio 1

Identificar un sistema de software existente en la comunidad (ej: app de biblioteca). Elaborar un informe que describa su **arquitectura de alto nivel** (componentes y relaciones).

Ejercicio 2

Diferenciar con un caso práctico la **arquitectura** y el **diseño** de un sistema web para tienda en línea. Presentar en dos documentos separados.

Ejercicio 3

Simular el rol de arquitecto en un sistema bancario digital. Identificar **atributos de calidad** (seguridad, confiabilidad, escalabilidad) y proponer soluciones arquitectónicas.